

Topic 9: Arrays

8.1 What is an Array?

- An array stores multiple values of the same type under one name
- Values are stored in contiguous (side-by-side) memory locations
- Access each value with its index — indexing starts at 0
- `int scores[5] = {90, 85, 78, 92, 88};`

8.2 Declaring and Initializing Arrays

- `int arr[5];` — declares, values are garbage (uninitialized)
- `int arr[5] = {1, 2, 3, 4, 5};` — declares and initializes
- `int arr[] = {1, 2, 3};` — size inferred from initializer (size = 3)
- Array size must be a compile-time constant in standard C++

8.3 Accessing and Modifying Elements

- `scores[0]` → first element (90)
- `scores[4]` → fifth element (88)
- `scores[2] = 95;` — modifies the third element
- Accessing out-of-bounds index is undefined behavior — no error, but dangerous

8.4 Arrays and Loops

```
int sum = 0;
for (int i = 0; i < 5; i++) {
    sum += scores[i];
}
cout << "Average: " << sum / 5.0 << endl;
```

- Always use `array_size - 1` as the last valid index

8.5 2D Arrays

```
int matrix[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
```

```
};  
cout << matrix[1][2]; // row 1, col 2 → 6
```

- Think of it as a table: first index = row, second index = column
- Nested loops to traverse: outer for rows, inner for columns

8.6 Array vs Individual Variables

- 5 individual variables: `int s1, s2, s3, s4, s5;` — hard to scale
 - Array of 5: `int s[5];` — easy to loop, easy to pass to functions
 - Arrays cannot change size at runtime — for dynamic sizes use vectors (C++ STL)
-